Unidade 1 – Ambiente de Trabalho para Programação Open BIM

Prof. Marco Aurélio Souza Mangan

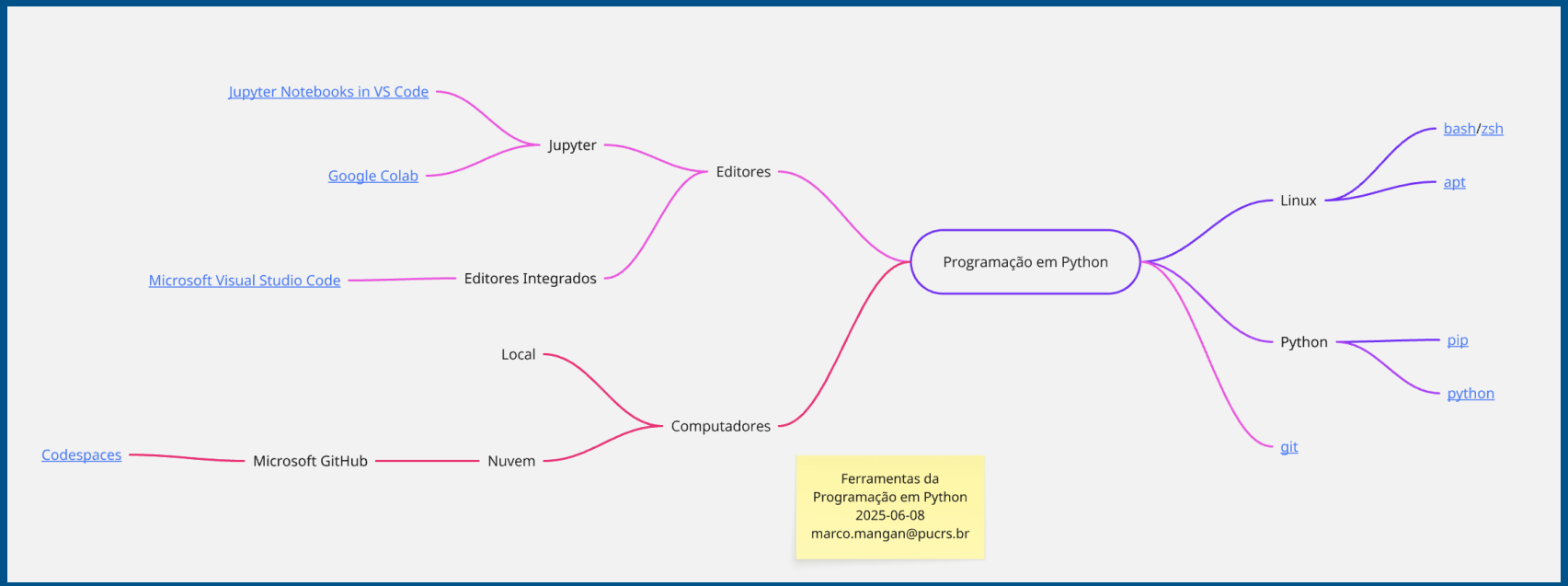
Objetivos da Unidade 1

- Apresentar ambientes de desenvolvimento para programação *Open Building Information Modeling (Open BIM)*
- Demonstrar como configurar um ambiente com Python
- Executar um primeiro script com Python e *Industry Foundation Classes (IFC)*

Pré-requisitos para Programar com Open BIM

- **Editor de código-fonte:**
  - Visual Studio Code (VS Code)
  - Jupyter Notebook
- **Computador Terminal e Python:**
  - Terminal disponível
  - Python instalado
  - Em alguns casos, permissão para instalar programas (admin)

Ferramentas da Programação em Python



Arquivo completo em:

`https://miro.com/app/board/uXjVIql0twY=/?share_link_id=545482922515`

Ambientes Disponíveis

- **Computador Local:** Pode ser necessário instalar VS Code e Python. Cuidado com backup de dados e de código-fonte!
- **Na nuvem, GitHub Codespaces:** VS Code e Python instalados. Ambiente em nuvem, integrado ao GitHub.
- **Na nuvem, Google Colab:** Uso de hipertextos na nuvem com acesso à Python em células.

Cuidado com privacidade e sigilo na nuvem!

Comparativo dos Ambientes Locais

Em geral

- Não depende de acesso constante à Internet
- Solução de backup deve ser escolhida
- Requer instalação de Python e alguma administração de sistema

VS Code no seu computador

- Requer instalação de VS Code

Jupyter Notebook no seu computador

- Requer instalação de Jupyter

Comparativo dos Ambientes na Nuvem

Em geral

- Depende de conexão com a Internet
- Python já disponível
- Disponível em qualquer navegador Web
- Código e dados são salvos na nuvem
- Dados na máquina virtual precisam ser salvos antes que ela seja liberada

Codespaces

- Utiliza VS Code
- Código e dados são versionados

Google Colab

- Utiliza Jupyter Notebook

Bibliotecas Relevantes

- `ifcopenshell` – leitura, modificação e escrita de arquivos IFC
- `numpy` e `pandas` – manipulação de dados
- `matplotlib` – gráficos (opcional)
- `pandas` - tabelas e séries de dados (opcional)
- demais bibliotecas do ecossistema de Python, ex.: streamlit (opcional)

Instalação das Bibliotecas

Exemplo de instalação: Google Colab

```
!pip install ifcopenshell pandas
```

Exemplo de requirements.txt para Codespaces:

```
ifcopenshell
pandas
```

Exemplo com IFC

- Exemplo, formado pelos arquivos `walls.ifc` e `count_walls.ifc`, para verificar a instalação correta de Python e das bibliotecas.
- O script `count_walls.py` conta quantas paredes estão descritas no arquivo `walls.ifc`.

walls.ifc

```
#13=IFCWALLSTANDARDCASE('22fj...',$,'Parede A'...);
#15=IFCWALLSTANDARDCASE('2zII...',$,'Parede B'...);
#16=IFCWALLSTANDARDCASE('0v7F...',$,'Parede C'...);
```

Arquivo completo em:
`https://raw.githubusercontent.com/bim-python-lab/bim-python-lab/refs/heads/main/unidade01/walls.ifc`

count_walls.py

```
import ifcopenshell

model = ifcopenshell.open("walls.ifc")
for wall in model.by_type("IfcWall"):
    print(wall.Name)
```

Demonstração no Google Colab

- Criar um caderno ativo no Google Colab
- Executar células em Python

Demonstração no GitHub

- Criar um repositório no GitHub
- Criar uma máquina virtual com o Codespaces
- Executar um script com um arquivo de dados

Repositório no GitHub com exemplos apresentados:

`https://github.com/bim-python-lab/bim-python-lab/`

Programação em Python

- Programação por Usuário Final
- Programação Assistida por Inteligência Artificial
- Comunidade de usuários: bibliotecas e exemplos

Como relatar um problema técnico

- Indique o ambiente usado: VS Code (Windows/Mac/Linux), Codespaces ou Colab
- Descreva o que você tentou fazer
- Copie e cole a mensagem de erro completa
- Anexe uma captura de tela, se possível

Captura de tela – Teclas de atalho:

- Windows: `Windows + Shift + S`
- macOS: `Command + Shift + 4`
- Linux (Gnome): `Print Screen` ou `Shift + Print Screen`
- Colab: use ferramenta nativa de captura do sistema

Encerramento da Unidade 1

- Escolha o ambiente que melhor se adapta à sua rotina
- Realize a instalação de bibliotecas
- Execute o script de exemplo
- Anote e relate qualquer erro ocorrido

Unidade 2 – Anatomia de um Script Open BIM com Python

Prof. Marco Aurélio Souza Mangan

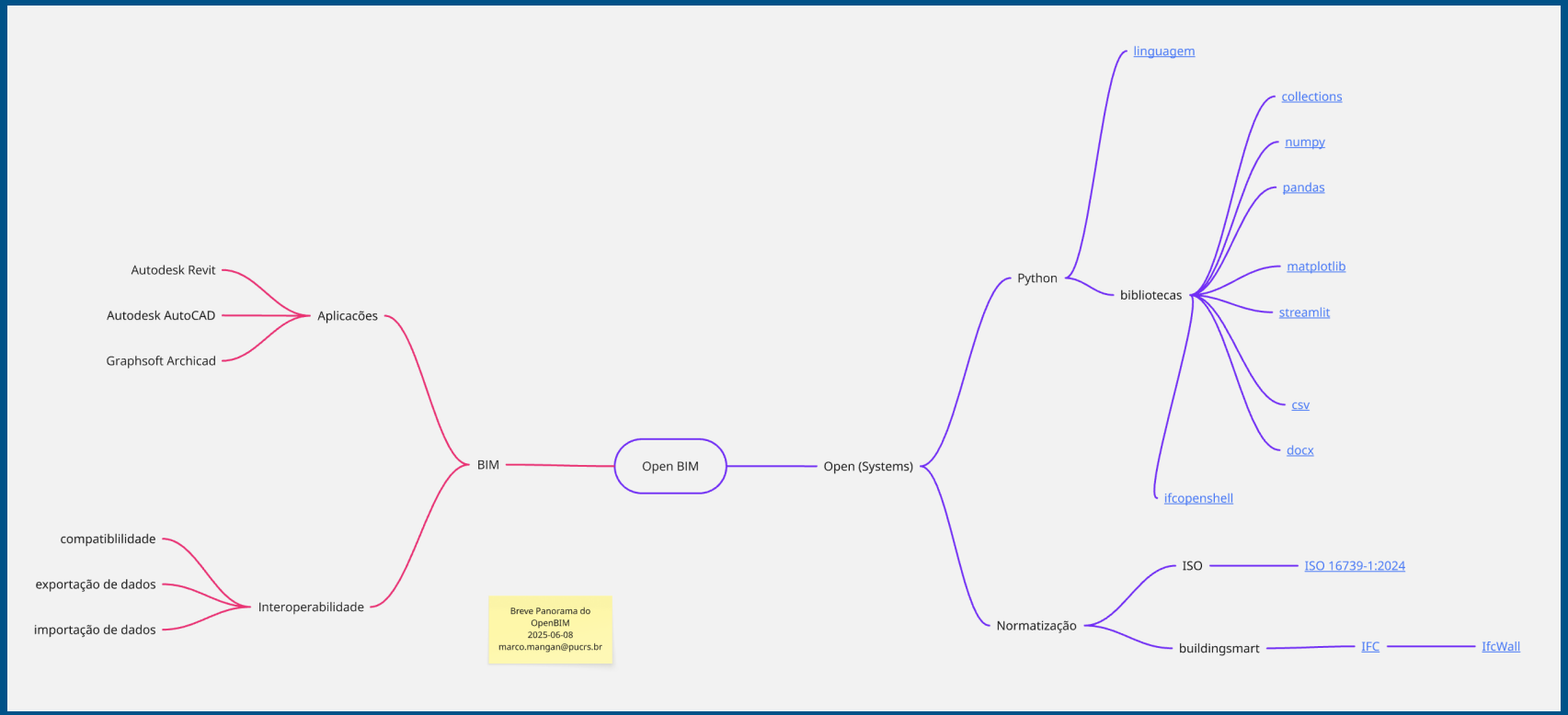
Objetivos da Unidade 2

- Entender como scripts Python manipulam arquivos IFC
- Apresentar as atividades comuns em scripts Open BIM
- Demonstrar exportação de dados e geração de gráficos

Por que usar Python para Open BIM?

- Linguagem simples, acessível e poderosa
- Facilita automação e extração de dados
- Integração com formatos abertos (ex: IFC)
- Uso crescente em scripts, plugins e validação

Breve Panorama do Open BIM



https://miro.com/app/board/uXjVIqmos90=/?share_link_id=165342835075

Cinco usos comuns de scripts Open BIM

1. Extração de dados e relatórios (CSV, DOCX, DataFrame)
2. Validação de modelos e checagem de regras
3. Classificação e agrupamento de elementos IFC
4. Geração e manipulação automática de objetos
5. Exportação e integração com outras ferramentas

Veremos alguns exemplos desses usos nesta unidade

O que você precisa saber de Python para Open BIM

- Importar bibliotecas com `import`
- Abrir arquivos IFC com o `ifcopenshell.open`
- Percorrer listas com `for`
- Acessar atributos com ponto (ex: `parede.Name`)
- Escrever arquivos com `open()`, `write()`, `csv.writer` etc.
- Transferir dados para outras bibliotecas como pandas, matplotlib e python-docx

count_elements.py

```
import ifcopenshell

model = ifcopenshell.open("building_elements.ifc")
types = ["IfcWall", "IfcDoor", "IfcWindow"]

for type in types:
    elems = model.by_type(type)
    print(f"{type}: {len(elems)} encontrados")
    for elem in elems:
        print(" -", elem.Name)
```

export_elements_to_csv.py

```
import ifcopenshell
import csv

model = ifcopenshell.open("building_elements.ifc")
types = ["IfcWall", "IfcDoor", "IfcWindow"]

with open("output.csv", "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["Tipo", "GlobalId", "Nome"])

    for t in types:
        for elem in model.by_type(t):
            writer.writerow([t, elem.GlobalId, elem.Name])
```

Elementos IFC comuns em projetos de arquitetura

- **IfcWall, IfcWallStandardCase** – paredes
- **IfcSlab** – lajes e pisos
- **IfcRoof** – coberturas
- **IfcDoor, IfcWindow** – portas e janelas
- **IfcStair, IfcRamp** – escadas e rampas
- **IfcSpace** – ambientes internos
- **IfcBuildingStorey, IfcBuilding, IfcSite** – pavimento, edifício, terreno

Variação na complexidade dos arquivos IFC

- Arquivos IFC variam conforme o tipo, detalhamento e fase do projeto

Estimativas típicas:

- Casa unifamiliar: 300–2.000 entidades
- Prédio residencial: 5.000–30.000 entidades
- Centro comercial: 30.000–100.000+ entidades
- Refinaria / planta industrial: 100.000–1.000.000+ entidades

Entidades que impactam no tamanho do modelo IFC

- **IfcProduct**: elementos físicos como paredes, portas, janelas, colunas etc.
- **IfcRel***: relacionamentos como agregação, atribuição, conexão e associação
- **IfcPropertySet**: grupo de propriedades atribuídas a elementos (ex: altura, material)
- **IfcRepresentation**: definição geométrica dos elementos (formas, sólidos, curvas)

Essas entidades são responsáveis por grande parte do volume informacional do modelo.

Automação de tarefas

- Tarefas recorrentes ou trabalhosas podem ser automatizadas
- Extração de informações, contagens de itens, cálculo de totais
- Os modelos são revisados continuamente, mas refazer tarefas a cada revisão está sujeito a erros e alto custo
- Automação auxilia a padronizar a operação, reduzir prazos e deslocar a atenção para tarefas criativas e de tomada de decisão

Repositórios abertos com IFC e scripts

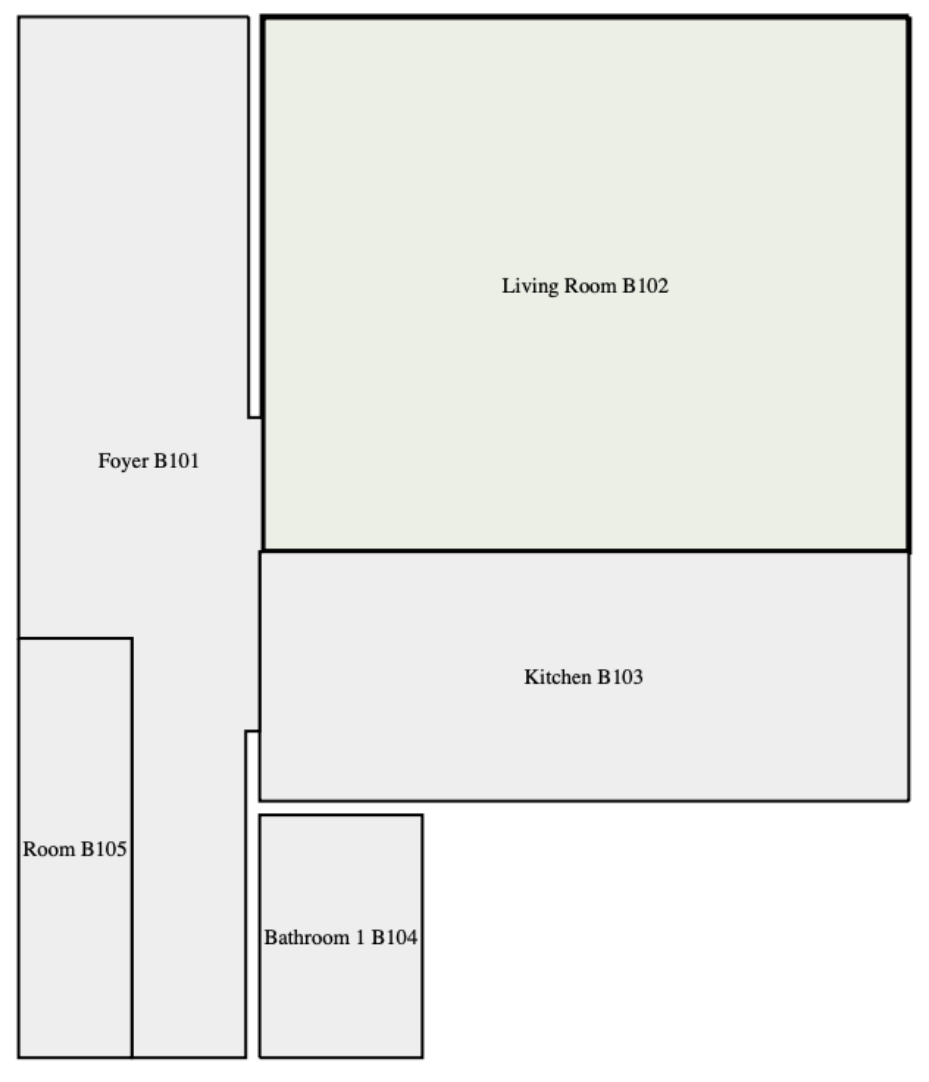
- **IfcOpenShell examples** – exemplos:
  `https://docs.ifcopenshell.org/ifcopenshell-python.html`
- **OSArch Test Models** – modelos abertos variados:
  `https://github.com/OpeningDesign/`
- **buildingSMART Samples** – arquivos IFC de referência:
  `https://github.com/buildingSMART/Sample-Test-Files`
- **BOT Duplex Apartment**
  `https://github.com/MadsHolten/BOT-Duplex-house/`

Um modelo maior: Duplex com elementos diversos

- Casa duplex com paredes, portas, janelas, escadas e espaços
- Um arquivo IFC com aprox. 2 MB
- `https://github.com/MadsHolten/BOT-Duplex-house/`
- `https://madsholten.github.io/BOT-Duplex-house/`
- `https://raw.githubusercontent.com/MadsHolten/BOT-Duplex-house/refs/heads/master/Model%20files/IFC/Duplex.ifc`

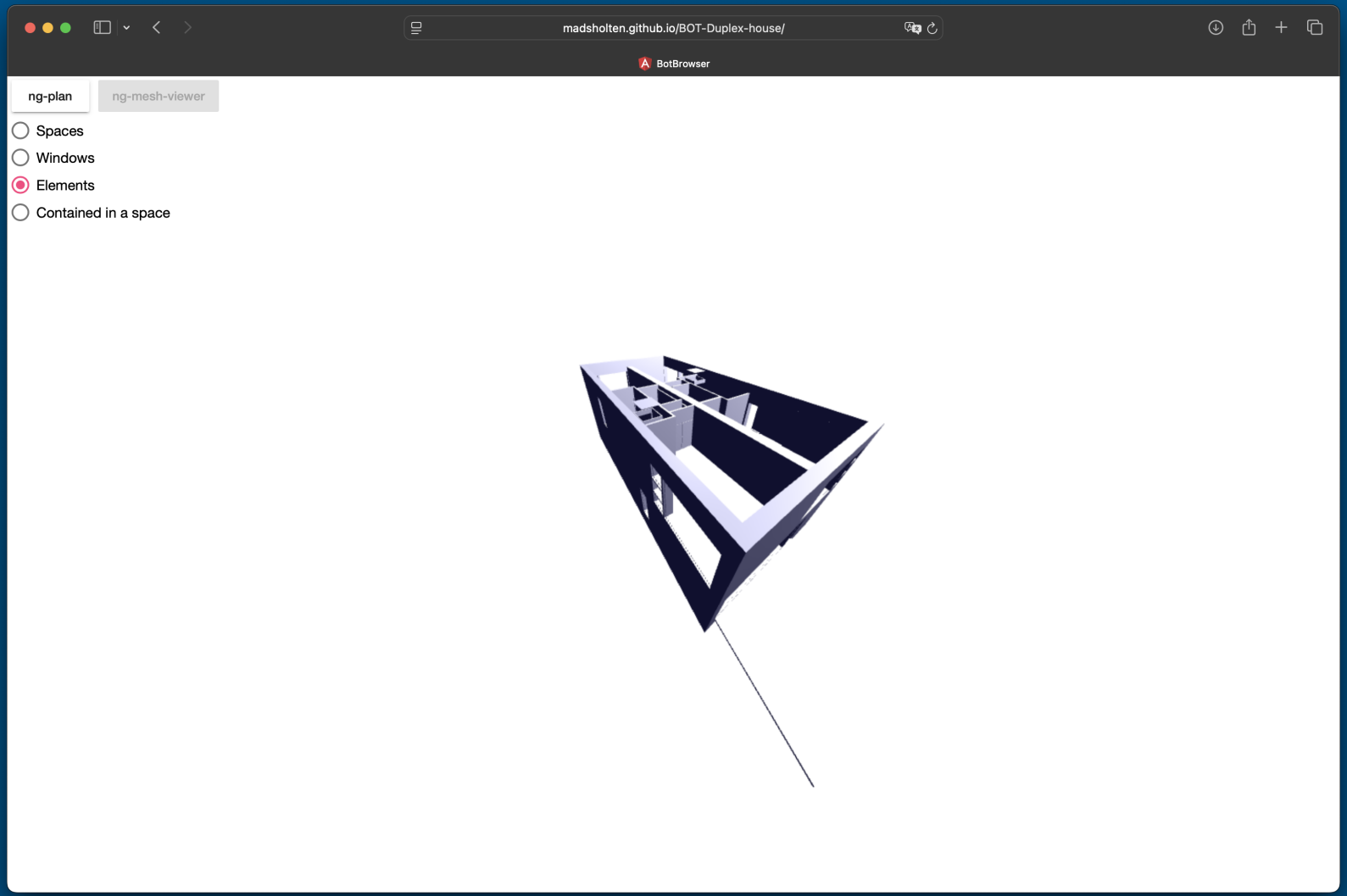
Vamos utilizar o arquivo IFC do Duplex.

Um modelo maior: Duplex com elementos diversos



Fonte: OSArch Test Models (OpeningDesign)

O Duplex em 3D



Versão interativa:

`https://madsholten.github.io/BOT-Duplex-house/`

CSV: Comma Separated Values

- A biblioteca CSV permite manipular dados em formato delimitado
- O formato CSV pode ser utilizado para importar e exportar dados de planilhas eletrônicas e sistemas de informação
- Tarefa: exportar dados de um IFC para importação em uma planilha eletrônica

CSV: duplex_list_elements.py

```
import ifcopenshell
import csv

model = ifcopenshell.open("duplex.ifc")
types = ["IfcWall", "IfcDoor", "IfcWindow",
         "IfcSlab", "IfcRoof", "IfcStair", "IfcSpace"]

with open("duplex_output.csv", "w", newline="") as csvfile:
    writer = csv.writer(csvfile)
    writer.writerow(["Tipo", "GlobalId", "Nome"])

    for t in types:
        for elem in model.by_type(t):
            name = elem.Name if hasattr(elem, "Name") else ""
            writer.writerow([t, elem.GlobalId, name])
```

duplex_output.csv

```
Tipo,GlobalId,Nome
IfcWall,2O2Fr$t4X7Zf8NOew3FKau,Basic Wall:Party Wall - CMU Residential Unit Dimising Wall:143239
IfcWall,2O2Fr$t4X7Zf8NOew3FNld,Basic Wall:Interior - Partition (92mm Stud):138584
IfcWall,2O2Fr$t4X7Zf8NOew3FNbT,Basic Wall:Party Wall - CMU Residential Unit Dimising Wall:139234
...
```

Pandas: duplex_to_dataframe.py

- A biblioteca Pandas oferece estruturas que permitem manipular grandes volumes de dados de forma eficiente
- Neste exemplo, um dataframe de Pandas é utilizado para organizar os dados e gerar o arquivo CSV
- A segunda versão do exemplo apresenta vantagens em modelos maiores e consome menos memória
- Mesma tarefa, exportar dados para planilha, porém com biblioteca pandas

pandas: Análise de Dados

```
import ifcopenshell
import pandas as pd

model = ifcopenshell.open("duplex.ifc")
types = ["IfcWall", "IfcDoor", "IfcWindow", "IfcSlab"]
data = []
for type in types:
    for elem in model.by_type(type):
        data.append({
            "Typo": type,
            "GlobalId": elem.GlobalId,
            "Name": getattr(elem, "Name", ""),
            "Tag": getattr(elem, "Tag", "")
        })
df = pd.DataFrame(data)
print(df.head())
df.to_csv("duplex_dataframe.csv", index=False)
```

duplex_to_dataframe_better.py

```
import ifcopenshell
import pandas as pd

model = ifcopenshell.open("duplex.ifc")
types = ["IfcWall", "IfcDoor", "IfcWindow"]
# generator - less memory required
data = (
    {
        "Type": type,
        "GlobalId": elem.GlobalId,
        "Nome": getattr(elem, "Name", ""),
        "Tag": getattr(elem, "Tag", "")
    }
    for type in types
    for elem in model.by_type(type)
)
df = pd.DataFrame.from_records(data)
print(df.head())
df.to_csv("duplex_dataframe.csv", index=False)
```

matplotlib: Gráficos Estatísticos

- Biblioteca que oferece recursos para gerar gráficos, diretamente em Python
- Tarefa: gerar gráficos para análise visual de dados que constam no IFC

matplotlib e collections: duplex_pizza_matplotlib.py

```
import ifcopenshell
from collections import Counter
import matplotlib.pyplot as plt

model = ifcopenshell.open("Duplex.ifc")
types = [e.is_a() for e in model if hasattr(e, "Name")]
contagem = Counter(types)

top = contagem.most_common(4)
labels = [item[0] for item in top]
data = [item[1] for item in top]

plt.pie(data, labels=labels, autopct="%1.1f%%", startangle=90)
plt.title("Distribuição dos tipos no modelo Duplex")
plt.axis("equal")
plt.savefig("pizzaplt")
plt.show()
```

matplotlib e pandas: duplex_pizza_df.py

```
import pandas as pd
import matplotlib.pyplot as plt

# Supondo que df foi gerado anteriormente
df = pd.read_csv("duplex_dataframe.csv")

contagem = df["Tipo"].value_counts().head(4)
contagem.plot.pie(autopct="%1.1f%%", startangle=90)
plt.title("Distribuição dos tipos no modelo Duplex")
plt.ylabel("")
plt.savefig("pizzadf")
plt.show()
```

python-docx

- Permite modificar e criar arquivos em formato .docx
- Permite gerar e atualizar documentos
- Bibliotecas similares podem trabalhar com outros formatos de texto como HTML e PDF.
- Tarefa: gerar relatórios com dados atualizados a partir do arquivo IFC mais recente

duplex_to_docx.py

```
import ifcopenshell
from docx import Document

model = ifcopenshell.open("duplex.ifc")
types = ["IfcWall", "IfcDoor", "IfcWindow"]

doc = Document()
doc.add_heading("Relatório IFC Modelo Duplex", 0)

for tipo in types:
    elementos = model.by_type(tipo)
    doc.add_heading(f"{tipo}: {len(elementos)} elementos", level
    for elem in elementos:
        nome = getattr(elem, "Name", "")
        gid = elem.GlobalId
        doc.add_paragraph(f"{nome} - {gid}")

doc.save("duplex_docx.docx")
```

Revisão: recursos de Python utilizados

- `for` loops e listas de dicionários
- Expressões geradoras com múltiplos `for`
- `getattr()` para acessar atributos com valor padrão
- Escrita e leitura de arquivos com `open()`
- Módulo `collections.Counter`

Módulo `csv` – principais métodos

- `csv.writer(csvfile)` – cria objeto de escrita
- `writer.writerow([...])` – escreve uma linha
- `writer.writerows([...])` – escreve várias linhas

Módulo python-docx – principais métodos

- `Document()` – cria documento novo
- `add_heading(texto, nível)` – títulos
- `add_paragraph(texto)` – parágrafos
- `save("arquivo.docx")` – salva o documento

Módulo `matplotlib.pyplot` – principais métodos

- `plt.pie()` – gráfico de pizza
- `plt.bar()`, `plt.barh()` – gráfico de barras
- `plt.savefig("figura.png")` – salvar imagem
- `plt.show()` – exibir gráfico

Biblioteca pandas – principais métodos

- `pd.DataFrame(...)` – cria tabela
- `df.to_csv()`, `df.to_excel()` – exporta
- `df.groupby()`, `df["coluna"].value_counts()` – agregações
- `df.plot.pie()`, `df.plot.bar()` – gráficos

Referência complementar: SciPy Lecture Notes

SciPy Lecture Notes – Guia completo para ciência de dados com Python

Bibliotecas abordadas:

- numpy, matplotlib, pandas, sympy, scipy
- Visualização de dados, ajuste de curvas, manipulação vetorial

`https://scipy-lectures.org/`

Prática de Programação

- Aprender a programar exige prática regular
- Recomendamos resolver problemas semanalmente

Plataformas sugeridas:

- BeeCrowd, CodeWars, HackerRank, LeetCode
- Kaggle – conjuntos de dados reais, notebooks e rankings

Escolha uma plataforma e resolva 1 problema por semana!

Existe um Kaggle para BIM?

Ainda não temos uma plataforma como Kaggle ou BeeCrowd voltada exclusivamente para BIM, mas há comunidades e repositórios ativos:

Comunidades e desafios BIM:

- OSArch: comunidade open source (`https://community.osarch.org`)
- GitHub: `IfcOpenShell`, `OpeningDesign`,
- buildingSMART: desafios e casos de uso internacionais

Encerramento da Unidade 2

- Escreva scripts Python que interagem com arquivos IFC
- Aproveite os exemplos de como ler, percorrer e extrair dados de elementos BIM
- Gere gráficos e documentos com base em dados do modelo IFC
- Aplique os conceitos a um modelo de uma construção de seu interesse
- Consulte a documentação e exemplos das bibliotecas e aprimore seus scripts

Unidade 3 – Padrões Reutilizáveis e Dimensões do Open BIM

Prof. Marco Aurélio Souza Mangan

Padrão comum em scripts BIM

A maioria dos scripts segue uma estrutura básica:

1. Carregar o modelo IFC
2. Filtrar os elementos desejados
3. Extrair e/ou modificar propriedades
4. Gerar saída (ex: CSV, gráfico, anotação, relatório)

Padrões reutilizáveis:

- Funções de filtragem
- Funções de anotação (‘pset‘)

Filtragem

```
def filter(model, type=None, named=None):
    for elem in model.by_type(tipo) if type else model:
        if named and named.lower() not in getattr(elem,
        "Name", "").lower():
            continue
        yield elem
```

Boas práticas:

- Comando yield interage com o comando for para retornar cada elemento filtrado

Anotação

```
def add_prop(model, pset, name, value):
     ...
    prop = model.create_entity("IfcPropertySingleValue", Name=name,...)
     ...
```

Boas práticas:

- Rotina auxilia na modificação de valores existentes
- Rotina simplifica o processo de operar sobre o conjunto de propriedades

Dimensões do BIM (3D a 8D)

- **3D** – Modelo geométrico (formas, volumes, tipos)
- **4D** – Tempo (cronogramas, sequenciamento de obra)
- **5D** – Custo (orçamentos, quantitativos, insumos)
- **6D** – Sustentabilidade (eficiência energética, impacto)
- **7D** – Operação e manutenção (O&M)
- **8D** – Segurança do trabalho (zonas de risco, EPIs)

Cada dimensão amplia o uso do modelo BIM ao longo do ciclo de vida.

Exemplo 3D – Contagem de elementos (paredes)

```
walls = list(filter(model, type="IfcWall"))
print("Paredes:", len(walls))
```

Boas práticas:

- Comando yield interage com o comando for para retornar cada elemento filtrado

Exemplo 4D – Etapas construtivas (paredes)

```
for wall in walls:
    phase = "Alvenaria Externa" if "exterior" in wall.Name.lower() else "Alvenaria Interna"
    pset = run("pset.add_pset", model, product=wall, name="PSet_Tarefa4D")
    add_prop(model, pset, "EtapaConstrutiva", phase)
    print(f"Valor {phase} adicionado à parede {wall.Name}")
```

Boas práticas:

- Nome padronizado de propriedades: `PSet_Tarefa4D`
- Uso de uma função genérica e reutilizável para adicionar propriedades

Exemplo 5D – Custo unitário por tipo de porta

```
for door in filter(model, tipo="IfcDoor"):
    value = 1800 if "dupla" in getattr(door, "Name", "").lower() else 1200
    pset = run("pset.add_pset", model, product=door, name="PSet_Custo5D")
    add_prop(model, pset, "CustoUnitario", value)
    print(f"Valor {value} adicionada à porta {door.Name}")
```

Boas práticas:

- Estrutura clara e reutilizável para diferentes tipos de elementos
- Uso de valores dinâmicos com base no nome do elemento

Exemplo 6D – Classificação energética de paredes

```
for wall in walls:
    ec = "B" if "exterior" in getattr(wall, "Name", "").lower() else "A"
    pset = run("pset.add_pset", model, product=wall, name="PSet_Energia6D")
    add_prop(model, pset, "ClasseEnergetica", ec)
    print(f"Valor {ec} adicionada à parede {wall.Name}")
```

Boas práticas:

- Anotação baseada em critérios técnicos ou espaciais
- Possibilidade de expansão com base em valores de transmitância

Exemplo 7D – Revisão e manutenção (lajes)

```
for slab in filter(model, tipo="IfcSlab"):
    pset = run("pset.add_pset", model, product=slab, name="PSet_
    deadline = "2026-01-01"
    add_prop(model, pset, "RevisaoPrevista", deadline)
    print(f"Valor {deadline} adicionada ao piso {slab.Name}")
```

Boas práticas:

- Anotação de datas facilita integração com cronogramas de manutenção

Exemplo 8D – Zonas de risco em espaços

```
for space in filter(model, tipo="IfcSpace"):
    pset = run("pset.add_pset", model, product=space, name="PSet_Seguranca8D")
    risk = "Risco de Queda" if "A105" in getattr(space, "Name", "").upper() else "Área Geral"
    add_prop(model, pset, "ZonaDeRisco", risk)
    print(f"Valor {risk} adicionada ao espaço {space.Name}")
```

Boas práticas:

- Aplicação semântica: significado da propriedade ligado ao contexto
- Ideal para controle visual e segurança em modelo federado

Consulta às propriedades

- Verifique query_duplex.py e query_walls.py para exemplos de consulta às propriedades originais e adicionadas.

query_duplex.py

```
def list_prop(elem):
    if hasattr(elem, "IsDefinedBy"):
        for rel in elem.IsDefinedBy:
            if rel.is_a("IfcRelDefinesByProperties"):
                pset = rel.RelatingPropertyDefinition
                if pset.is_a("IfcPropertySet"):
                    print(f"  PSet: {pset.Name}")
                    for prop in pset.HasProperties:
                        print(f"    {prop.Name}: {prop.NominalValue.wrappedValue}")
```

Boas práticas:

- Utiliza as relações do modelo para descobrir todas as propriedades existentes

Encerramento da Unidade 3

- Explorar as dimensões do BIM de 3D a 8D
- Aplicar padrões reutilizáveis em scripts Python
- Utilizar funções genéricas de filtragem e anotação
- Produzir exemplos de propriedades com foco em geometria, tempo, custo, sustentabilidade, manutenção e segurança
- Consolidar um script completo que aplica todas as anotações no modelo `duplex.ifc`

Cada execução imprime no terminal a operação realizada.